

Solution Requirements

Date	03 July 2026
Team ID	
Project Name	Credit Card Approval Prediction
Maximum Marks	4 Marks

Define Functional and Non-Functional Requirements:

Create a comprehensive list of requirements to define exactly what your solution will do and how well it will perform.

Functional Requirements (FR) define specific behaviors, functions, or features of the system (What the system should *do*).

Non-Functional Requirements (NFR) specify the criteria that judge the operation of a system, rather than specific behaviors (How the system should *be*).

Fill out the descriptions and necessary details for each category provided below.

Step 1: Functional Requirements (FR)

S.No	Requirement Category	Requirement Description	Priority (High/Medium/Low)
1	Authentication	Users access the Flask web application directly via browser without login. The system identifies user type (single applicant or compliance officer) based on the page they access (/ for single, /batch for batch screening). IBM Watson ML API uses API key authentication for cloud endpoint access.	High
2	Authorization Levels	Three user roles are supported: • Credit Analyst — accesses single prediction form (/predict route) • Compliance Officer — accesses batch CSV screening (/batch route) • Data Engineer/ML Engineer — accesses Jupyter notebooks and model files in VS Code	High
3	External Interfaces	• Kaggle API — for downloading application_record.csv and credit_record.csv datasets • IBM Watson Machine Learning API — for cloud-hosted model deployment and scoring • Flask Web Framework — serves the prediction UI and REST endpoints • XGBoost/Scikit-learn — for model training and inference	High
4	Transactions Processing	• Single prediction: User submits form → Flask collects 17 input fields → pre-processes into 47 model features → XGBoost predicts → result displayed instantly • Batch prediction: User uploads CSV → Flask reads each row → runs model on all rows → returns results table with applicant ID, approval status, and risk score %	High

S.No	Requirement Category	Requirement Description	Priority (High/Medium/Low)
5	Reporting	• Single prediction report: Displays approval/rejection result and risk score % per applicant • Batch screening report: Table showing all applicant IDs, results (green=approved, red=rejected), and individual risk scores • Model comparison chart: Bar chart comparing accuracy, precision, recall, F1 for all 4 models • EDA visualizations: Income distribution, age distribution, class imbalance, correlation heatmap	Medium
6	Business Rules etc.	• An applicant is labeled High Risk (Rejected) if STATUS code is 1, 2, 3, 4, or 5 (30+ days overdue) in any month of credit history • An applicant is labeled Low Risk (Approved) if STATUS is always C (paid) or X (no loan) • DAYS_EMPLOYED value of 365243 indicates unemployed/retired — treated as IS_UNEMPLOYED=1 • Missing OCCUPATION_TYPE values are filled with 'Unknown' category • Class imbalance (88%/12%) is handled using class_weight='balanced' in all models	High
7	Compliance to Laws or Regulations	• Data Privacy: Applicant financial data is processed locally and not stored permanently beyond the session — no personal data is retained in the Flask application • IBM Watson ML: Follows IBM Cloud data residency and security compliance policies • The system does not collect, store, or share Aadhaar, PAN, or any government ID data • Predictions are advisory only — final credit decisions remain with the bank analyst	High
8	Other	• Batch CSV upload must follow the required column format: applicant_id, gender, own_car, own_realty, children, income, income_type, education, family_status, housing_type, occupation, age, years_employed, family_members, work_phone, phone, email • The system must display a downloadable sample CSV template for batch screening • The Flask application must run on Python 3.x with all dependencies installable via pip	Medium

Step 2: Non-Functional Requirements (NFR)

S.No	NFR Category	Requirement Description	Target Metric / Acceptance Criteria
1	Performance & Speed	The Flask web application must return a single applicant prediction result within 2 seconds of form submission. Batch CSV processing for up to 100 applicants must complete within 10 seconds.	Single prediction < 2 sec; Batch (100 rows) < 10 sec
2	Scalability	The IBM Watson Machine Learning deployment pipeline must support cloud-hosted real-time inference, enabling the model to scale to handle multiple concurrent prediction requests without performance degradation.	Supports concurrent requests via IBM Watson ML cloud endpoint

S.No	NFR Category	Requirement Description	Target Metric / Acceptance Criteria
3	Security & Data Privacy	Applicant financial data submitted via the web form is not stored in any database or log file. IBM Watson ML API key is stored securely and not exposed in the frontend. No sensitive PII is transmitted beyond the local prediction pipeline.	No persistent data storage; API key secured server-side
4	Reliability & Availability	The local Flask application must start successfully every time with py app.py command and handle invalid or missing form inputs gracefully without crashing. The IBM Watson cloud endpoint must maintain high availability for production use.	Zero crashes on valid input; 99.9% uptime for Watson endpoint
5	Usability & Accessibility	The web application must have a clean, intuitive HTML form interface accessible via any modern browser (Chrome, Firefox, Edge). Labels for all form fields must be clear. Results must be displayed in plain English — not raw model output codes.	Works on Chrome, Firefox, Edge; result shown in plain English
6	Other (Maintainability & Portability)	• Maintainability: All 5 Jupyter notebooks are clearly structured by project phase (data exploration, EDA, preprocessing, model building, Watson deployment) for easy review • Portability: Project runs on Windows, macOS, and Linux with Python 3.x and pip • Model Reproducibility: random_state=42 is used in all models for consistent results • Model file (credit_approval_model.pkl) is self-contained and portable across environments	Runs on any OS with Python 3.x; random_state=42 for reproducibility